

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report
from sklearn.model_selection import GridSearchCV
from imblearn.over_sampling import SMOTE
from sklearn.cluster import KMeans
```

```
from google.colab import files
uploaded = files.upload()
```

 Choose Files churn-bigml-20.csv

- churn-bigml-20.csv(text/csv) - 56329 bytes, last modified: 7/16/2025 - 100% done

Saving churn-bigml-20.csv to churn-bigml-20 (1).csv

```
import pandas as pd
df = pd.read_csv('churn-bigml-20.csv')
df
```



	State	Account length	Area code	International plan	Voice mail plan	Number vmail messages	Total day minutes	Total day calls	Total day charge	Total eve minutes	Total eve calls	Total eve charge	Total night minutes	Total night calls	Total night charge	mi
0	LA	117	408	No	No	0	184.5	97	31.37	351.6	80	29.89	215.8	90	9.71	
1	IN	65	415	No	No	0	129.1	137	21.95	228.5	83	19.42	208.8	111	9.40	
2	NY	161	415	No	No	0	332.9	67	56.59	317.8	97	27.01	160.6	128	7.23	
3	SC	111	415	No	No	0	110.4	103	18.77	137.3	102	11.67	189.6	105	8.53	
4	HI	49	510	No	No	0	119.3	117	20.28	215.1	109	18.28	178.7	90	8.04	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
662	WI	114	415	No	Yes	26	137.1	88	23.31	155.7	125	13.23	247.6	94	11.14	
663	AL	106	408	No	Yes	29	83.6	131	14.21	203.9	131	17.33	229.5	73	10.33	
664	VT	60	415	No	No	0	193.9	118	32.96	85.0	110	7.23	210.1	134	9.45	
665	WV	159	415	No	No	0	169.8	114	28.87	197.7	105	16.80	193.7	82	8.72	
666	CT	184	510	Yes	No	0	213.8	105	36.35	159.6	84	13.57	139.2	137	6.26	

667 rows x 20 columns

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
# Data Exploration
print("Dataset shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nData types and missing values:")
print(df.info())
print("\nSummary statistics:")
print(df.describe())
print("\nChurn distribution:")
print(df['Churn'].value_counts(normalize=True))
```



mean	100.537031	30.701709	203.355322	100.470702
std	20.396790	9.436463	49.719268	18.948262
min	30.000000	4.400000	48.100000	37.000000
25%	87.500000	24.860000	171.050000	88.000000
50%	101.000000	30.310000	203.700000	101.000000
75%	115.000000	37.520000	236.450000	113.000000
max	165.000000	56.830000	361.800000	168.000000

	Total eve charge	Total night minutes	Total night calls \
count	667.000000	667.000000	667.000000
mean	17.285262	199.685307	100.113943
std	4.226160	49.759931	20.172505
min	4.090000	23.200000	42.000000
25%	14.540000	167.950000	86.000000
50%	17.310000	201.600000	100.000000
75%	20.095000	231.500000	113.500000
max	30.750000	367.700000	175.000000

	Total night charge	Total intl minutes	Total intl calls \
count	667.000000	667.000000	667.000000
mean	8.985907	10.238381	4.527736
std	2.239429	2.807850	2.482442
min	1.040000	0.000000	0.000000
25%	7.560000	8.600000	3.000000
50%	9.070000	10.500000	4.000000
75%	10.420000	12.050000	6.000000
max	16.550000	18.300000	18.000000

	Total intl charge	Customer service calls
count	667.000000	667.000000
mean	2.764948	1.563718
std	0.758167	1.333357
min	0.000000	0.000000
25%	2.320000	1.000000
50%	2.840000	1.000000
75%	3.255000	2.000000
max	4.940000	8.000000

Churn distribution:

Churn

False 0.857571

True 0.142429

Name: proportion, dtype: float64

Visualizations

plt.figure(figsize=(15, 10))

↗ <Figure size 1500x1000 with 0 Axes>
 ↗ <Figure size 1500x1000 with 0 Axes>

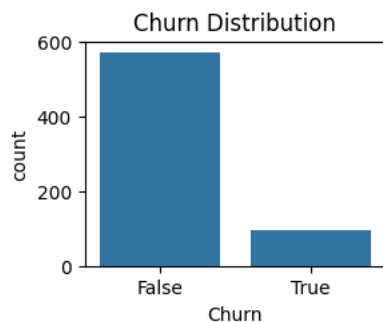
Churn distribution

plt.subplot(2, 2, 1)

sns.countplot(x='Churn', data=df)

plt.title('Churn Distribution')

↗ Text(0.5, 1.0, 'Churn Distribution')



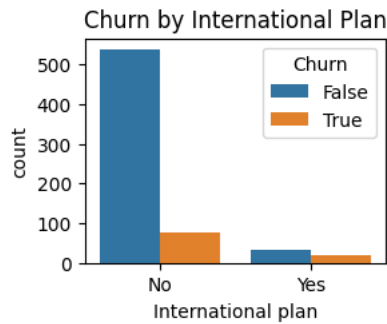
International plan vs churn

plt.subplot(2, 2, 2)

sns.countplot(x='International plan', hue='Churn', data=df)

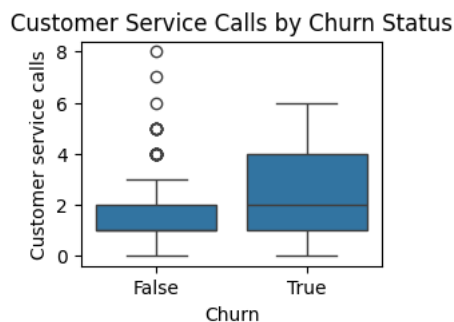
plt.title('Churn by International Plan')

```
Text(0.5, 1.0, 'Churn by International Plan')
```

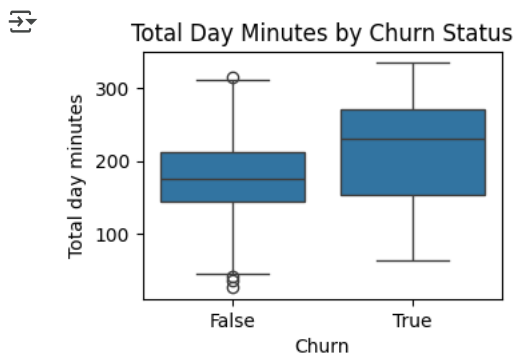


```
# Customer service calls vs churn
plt.subplot(2, 2, 3)
sns.boxplot(x='Churn', y='Customer service calls', data=df)
plt.title('Customer Service Calls by Churn Status')
```

```
Text(0.5, 1.0, 'Customer Service Calls by Churn Status')
```



```
# Total day minutes vs churn
plt.subplot(2, 2, 4)
sns.boxplot(x='Churn', y='Total day minutes', data=df)
plt.title('Total Day Minutes by Churn Status')
plt.tight_layout()
plt.show()
```



```
#Feature Engineering
# Create new features that might be predictive
df['Total minutes'] = df['Total day minutes'] + df['Total eve minutes'] + df['Total night minutes']
df['Total calls'] = df['Total day calls'] + df['Total eve calls'] + df['Total night calls']
df['Total charge'] = df['Total day charge'] + df['Total eve charge'] + df['Total night charge']
df['Avg minutes per call'] = df['Total minutes'] / df['Total calls']
df['Charge per minute'] = df['Total charge'] / df['Total minutes']

# Data Preprocessing
# Separate features and target
X = df.drop('Churn', axis=1)
y = df['Churn']

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
# Define categorical and numerical features
categorical_features = ['State', 'International plan', 'Voice mail plan']
numerical_features = [col for col in X.columns if col not in categorical_features + ['Area code']]
```

```

# Preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

# Handle class imbalance with SMOTE
smote = SMOTE(random_state=42)

# Model Building
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'SVM': SVC(probability=True, random_state=42),
    'KNN': KNeighborsClassifier()
}

# Evaluate models
results = {}
for name, model in models.items():
    # Create pipeline
    pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', model) # SMOTE will be applied outside the pipeline
    ])

    # Fit the pipeline on the training data
    X_train_processed = preprocessor.fit_transform(X_train)

    # Apply SMOTE to the preprocessed training data
    X_train_smote, y_train_smote = smote.fit_resample(X_train_processed, y_train)

    # Fit the classifier on the SMOTE-augmented training data
    pipeline.named_steps['classifier'].fit(X_train_smote, y_train_smote)

    # Make predictions
    y_pred = pipeline.predict(X_test)
    y_prob = pipeline.predict_proba(X_test)[: , 1]

    # Store results
    results[name] = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1 Score': f1_score(y_test, y_pred),
        'ROC AUC': roc_auc_score(y_test, y_prob)
    }

# Convert results to DataFrame
results_df = pd.DataFrame(results).T
print("\nModel Performance Comparison:")
print(results_df)

```

↗

Model Performance Comparison:					
	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.828358	0.428571	0.631579	0.510638	0.826087
Random Forest	0.932836	0.857143	0.631579	0.727273	0.920824
Gradient Boosting	0.970149	1.000000	0.789474	0.882353	0.908467
SVM	0.940299	0.823529	0.736842	0.777778	0.914416
KNN	0.798507	0.394737	0.789474	0.526316	0.854691

```

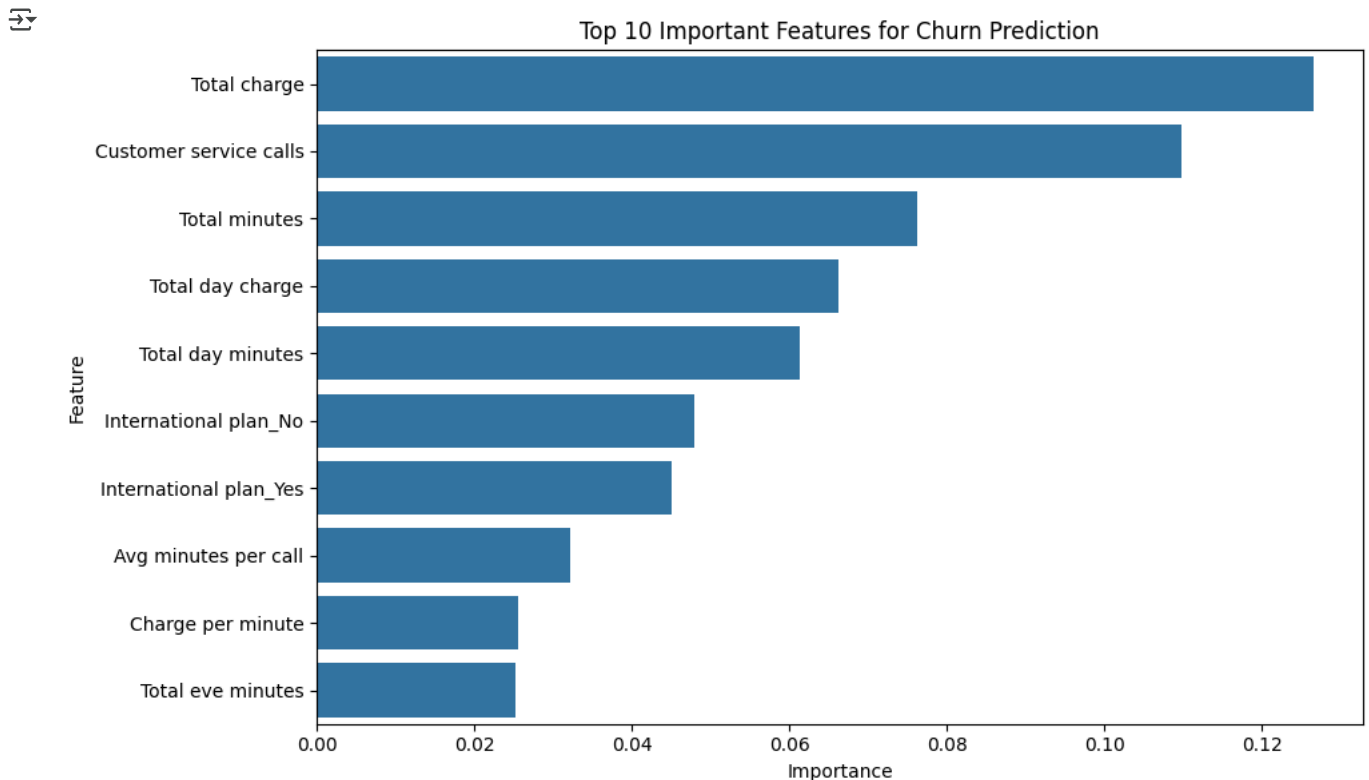
# Feature Importance Analysis
# Get feature names after one-hot encoding
preprocessor.fit(X_train)
cat_features = rf_pipeline.named_steps['preprocessor'].named_transformers_['cat'].get_feature_names_out(categorical_features)
feature_names = numerical_features + list(cat_features)

# Get feature importances from the Random Forest model
importances = rf_pipeline.named_steps['classifier'].feature_importances_

# Create a DataFrame for feature importance
feature_importance = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
feature_importance = feature_importance.sort_values('Importance', ascending=False)

```

```
# Plot top 10 most important features
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance.head(10))
plt.title('Top 10 Important Features for Churn Prediction')
plt.tight_layout()
plt.show()
```

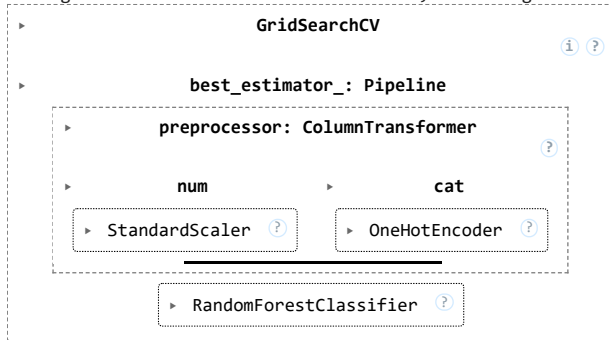


```
# Hyperparameter Tuning for Random Forest
param_grid = {
    'classifier__n_estimators': [100, 200],
    'classifier__max_depth': [None, 10, 20],
    'classifier__min_samples_split': [2, 5],
    'classifier__min_samples_leaf': [1, 2]
}
```

```
# Set up GridSearchCV
grid_search = GridSearchCV(
    estimator=rf_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=1
)
```

```
# Fit the grid search to the data
grid_search.fit(X_train, y_train)
```

Fitting 5 folds for each of 24 candidates, totalling 120 fits



```
# Print best parameters and score
print("\nBest parameters:", grid_search.best_params_)
print("Best ROC AUC score:", grid_search.best_score_)
```

```
Best parameters: {'classifier__max_depth': None, 'classifier__min_samples_leaf': 1, 'classifier__min_samples_split': 2, 'classifier__min_samples_split': 2, 'classifier__min_samples_split': 2}
Best ROC AUC score: 0.9109332696289218
```

```
# Evaluate the tuned model
best_model = grid_search.best_estimator_
y_pred_tuned = best_model.predict(X_test)
y_prob_tuned = best_model.predict_proba(X_test)[:, 1]
```

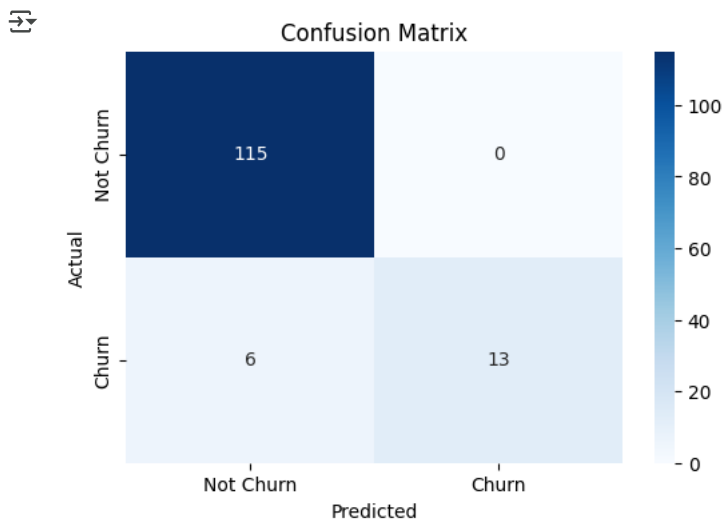
```
# Classification report
print("\nClassification Report for Tuned Model:")
print(classification_report(y_test, y_pred_tuned))
```

```
Classification Report for Tuned Model:
              precision    recall  f1-score   support

   False      0.95      1.00      0.97       115
    True      1.00      0.68      0.81        19

 accuracy      0.96
 macro avg      0.98
 weighted avg      0.96
```

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_tuned)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Churn', 'Churn'],
            yticklabels=['Not Churn', 'Churn'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```



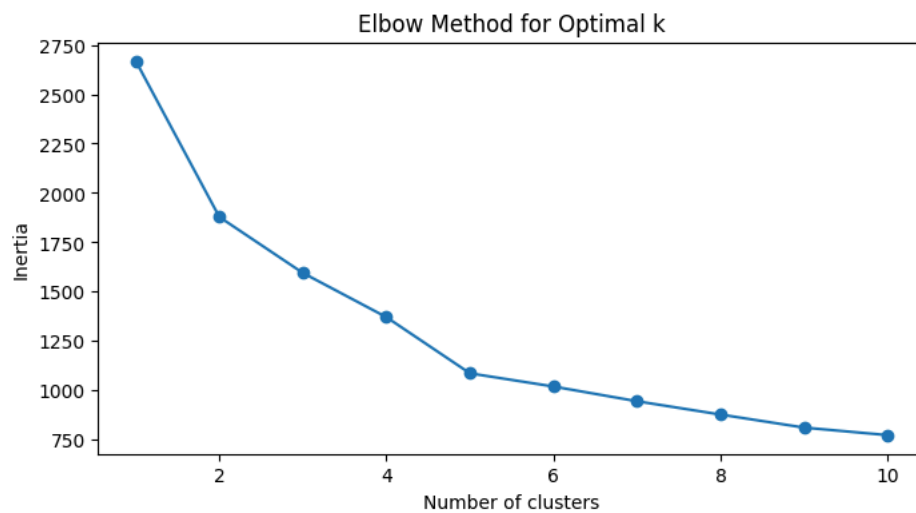
```
# Market Segmentation using Clustering
# Select features for clustering
cluster_features = ['Total minutes', 'Total charge', 'Customer service calls', 'Total intl minutes']
X_cluster = df[cluster_features]
```

```
# Standardize the features
scaler = StandardScaler()
X_cluster_scaled = scaler.fit_transform(X_cluster)
```

```
# Determine optimal number of clusters using elbow method
inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_cluster_scaled)
    inertia.append(kmeans.inertia_)
```

```
# Plot the elbow curve
plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters')
```

```
plt.ylabel('Inertia')
plt.show()
```



```
# Based on the elbow plot, choose optimal k (e.g., 3)
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
clusters = kmeans.fit_predict(X_cluster_scaled)
```

```
# Add clusters to original dataframe
df['Cluster'] = clusters
```

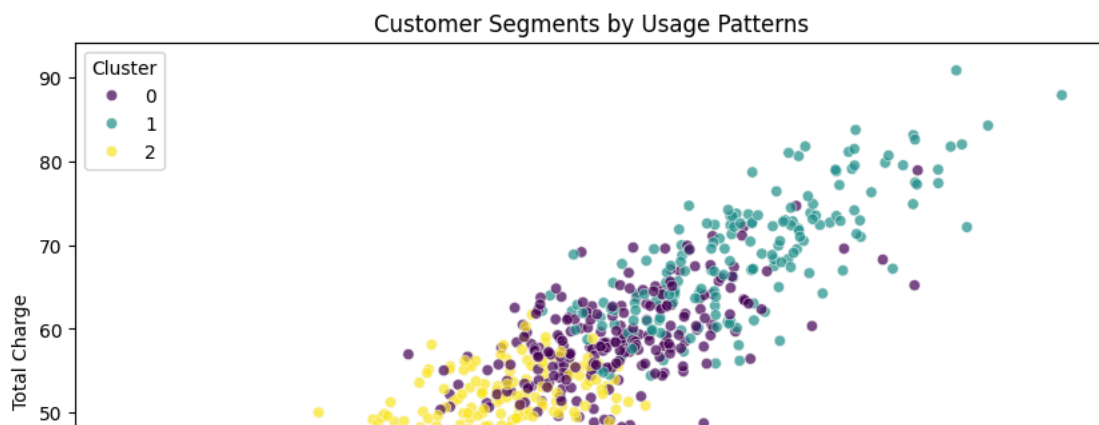
```
# Analyze cluster characteristics
# Select only numeric columns for aggregation
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
cluster_analysis = df.groupby('Cluster')[numeric_cols + ['Churn']].mean()
print("\nCluster Analysis:")
print(cluster_analysis[cluster_features + ['Churn']])
```



```
Cluster Analysis:
      Total minutes  Total charge  Customer service calls \
Cluster
0          599.535931      58.753247                0.965368
1          677.833520      68.422123                1.815642
2          504.651751      47.554125                1.926070

      Total intl minutes      Churn
Cluster
0          12.071429  0.056277
1           8.575419  0.279330
2           9.749027  0.124514
```

```
# Visualize clusters in 2D space
plt.figure(figsize=(10, 6))
sns.scatterplot(
    x='Total minutes',
    y='Total charge',
    hue='Cluster',
    data=df,
    palette='viridis',
    alpha=0.7
)
plt.title('Customer Segments by Usage Patterns')
plt.xlabel('Total Minutes')
plt.ylabel('Total Charge')
plt.show()
```



Visualization with customer service calls

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(
    x='Total minutes',
    y='Customer service calls',
    hue='Cluster',
    data=df,
    palette='viridis',
    alpha=0.7
)
```

```
plt.title('Customer Segments by Usage and Service Calls')
plt.xlabel('Total Minutes')
```

```
plt.ylabel('Customer Service Calls')
plt.show()
```

